

Tema 06 · JSP: reutilización con include y manejo de Session

Clase 020 (02-09) · Teoría: «JSP» (Clase020_jsp.pdf)

Programación Avanzada · Prof. Macrino Matías · Universidad del Salvador – Sede Pilar · 2.º cuatrimestre

1. Reutilizar partes de una página: include

En un sitio web muchas páginas comparten partes (la barra de navegación, el pie). En lugar de copiarlas en cada JSP, se escriben **una sola vez** en un archivo (common/navbar.jsp) y se **incluyen** en las demás. Hay dos formas:

	Include estático	Include dinámico
Sintaxis	<code><%@ include file="common/navbar.jsp" %></code> (directiva)	<code><jsp:include page="common/navbar.jsp" /></code> (acción)
Cuándo se incluye	En tiempo de traducción : el contenido se copia dentro del JSP antes de compilarlo (queda un solo servlet)	En tiempo de ejecución : en cada petición se ejecuta el otro JSP y se inserta su salida
Parámetros	No admite	Sí, con <code><jsp:param name="..." value="..." /></code> ; el JSP incluido los lee con <code>request.getParameter</code>
Cuándo conviene	Fragmentos fijos (cabecera, pie)	Fragmentos que cambian según la petición (menú con la opción activa)

2. El problema del estado: Session

HTTP es un protocolo **sin estado**: cada petición es independiente y el servidor «se olvida» del usuario entre una y otra. Para recordar quién es el usuario (carrito de compras, home banking, exámenes online, login) se usa la **Session**.

Cómo funciona: la primera vez que el usuario entra, el servidor crea un objeto **HttpSession** exclusivo para él y le envía un identificador (**JSESSIONID**) en una **cookie**. En cada petición siguiente el navegador devuelve esa cookie y el servidor recupera la sesión de ese usuario. Los datos se guardan **en el servidor** (en el cliente solo viaja el id).

Método de HttpSession (objeto implícito session)	Qué hace
setAttribute("clave", objeto)	Guarda un dato en la sesión
getAttribute("clave")	Lee un dato (devuelve Object: hay que castear; null si no existe)
removeAttribute("clave")	Borra un dato
invalidate()	Destruye toda la sesión (cerrar sesión / logout)
getId()	Devuelve el identificador de la sesión
isNew()	true si la sesión se acaba de crear
setMaxInactiveInterval(segundos)	Tiempo de inactividad tras el cual la sesión expira

Alcances (scopes) en una aplicación web, de menor a mayor vida: page (la página), request (una petición), session (el usuario durante su visita) y application (toda la aplicación, compartido por todos los usuarios).

Para el parcial: la Session guarda el usuario logueado y su perfil (admin o chofer); al cerrar sesión se hace **invalidate()** y se limpian las cookies. Ver «SesionUtil», «LoginServlet» y «LogoutServlet» de la solución del parcial.

Código de las clases, explicado línea por línea

Cada recuadro gris reproduce líneas reales del archivo (con su número de línea) y el recuadro verde de abajo explica qué hacen y por qué están escritas así.

Clase 020 - 02-09: include estático y dinámico, y contador con Session

Se arma un pequeño sitio con una **barra de navegación reutilizable** (common/navbar.jsp) incluida con **jsp:include** en dos páginas (inicio.jsp y contacto.jsp), pasándole **parámetros** para marcar la opción activa; y una página **session.jsp** con un contador guardado en la **Session**.

020 - Clase 02-09 / pom.xml

Qué es y para qué sirve: Archivo de configuración de Maven (POM) de una aplicación web: coordenadas, dependencias y nombre del WAR.

Cómo defenderlo / qué decir: Es el POM que genera el arquetipo de Maven para aplicaciones web. Lo importante: **packaging war**, las APIs de Servlet/JSP como **provided** (las aporta el contenedor) y, si la página usa JSTL, esas librerías sí van en el WAR.

```
1 | <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2 |   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
```

Líneas 1 a 2: Cabecera del POM: elemento raíz **project** con sus espacios de nombres XML (definen el formato válido del archivo).

```
3 |   <modelVersion>4.0.0</modelVersion>
```

Línea 3: Versión del modelo de POM (siempre 4.0.0).

```
4 |   <groupId>edu.usal</groupId>
5 |   <artifactId>clase-02-09</artifactId>
6 |   <packaging>war</packaging>
7 |   <version>1.0-SNAPSHOT</version>
8 |   <name>clase-02-09 Maven Webapp</name>
9 |   <url>http://maven.apache.org</url>
```

Líneas 4 a 9: Coordenadas del proyecto: **groupId** edu.usal (la organización), **artifactId** clase-02-09 (el nombre), **packaging war** (se empaqueta como aplicación web, un archivo .war que se despliega en Tomcat), versión 1.0-SNAPSHOT, nombre descriptivo y URL (datos del arquetipo maven-archetype-webapp).

```
10 |   <dependencies>
```

Línea 10: Abre la lista de dependencias (librerías que Maven descarga solo).

```
11 |     <dependency>
12 |       <groupId>junit</groupId>
13 |       <artifactId>junit</artifactId>
14 |       <version>3.8.1</version>
15 |       <scope>test</scope>
16 |     </dependency>
17 |
```

Líneas 11 a 17: **JUnit 3.8.1** con **scope test**: solo se usa para pruebas unitarias, no viaja dentro del WAR.

```
18 |     <dependency>
19 |       <groupId>javax.servlet</groupId>
20 |       <artifactId>javax.servlet-api</artifactId>
21 |       <version>4.0.1</version>
22 |       <scope>provided</scope>
23 |     </dependency>
24 |
```

Líneas 18 a 24: **javax.servlet-api 4.0.1** con **scope provided**: se necesita para compilar los servlets, pero en ejecución la provee el servidor (Tomcat 9), por eso no se incluye en el WAR.

```

25 | <dependency>
26 |   <groupId>javax.servlet.jsp</groupId>
27 |   <artifactId>javax.servlet.jsp-api</artifactId>
28 |   <version>2.3.3</version>
29 |   <scope>provided</scope>
30 | </dependency>

```

Líneas 25 a 30: javax.servlet.jsp-api 2.3.3, también **provided**: la API de JSP que aporta Tomcat.

```

31 | </dependencies>

```

Línea 31: Cierre de la lista de dependencias.

```

32 | <build>
33 |   <finalName>clase-02-09</finalName>
34 | </build>

```

Líneas 32 a 34: finalName define el nombre del WAR generado (clase-02-09.war) y por lo tanto la **ruta de contexto** de la aplicación: http://localhost:8080/clase-02-09/.

```

35 | </project>

```

Línea 35: Cierre del POM.

020 - Clase 02-09 / src / main / webapp / WEB-INF / web.xml

Qué es y para qué sirve: web.xml: descriptor de despliegue de la aplicación web. Vive en WEB-INF y es el archivo de configuración que lee el contenedor (Tomcat) al iniciar.

Cómo defenderlo / qué decir: Aquí solo contiene el nombre de la aplicación porque todavía no hay servlets declarados por XML. Es el lugar donde se declararían servlets, filtros, páginas de error y configuración de sesión (ver Tema 08).

```

1 | <!DOCTYPE web-app PUBLIC
2 |   "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
3 |   "http://java.sun.com/dtd/web-app_2_3.dtd" >
4 |

```

Líneas 1 a 4: Declaración del tipo de documento (**DOCTYPE**): indica que es un descriptor de despliegue de aplicación web.

```

5 | <web-app>

```

Línea 5: Elemento raíz **web-app** en su versión mínima (Servlet 2.3): no declara espacios de nombres ni versión.

```

6 |   <display-name>Archetype Created Web Application</display-name>

```

Línea 6: Nombre descriptivo de la aplicación (lo muestra el administrador de Tomcat).

```

7 | </web-app>

```

Línea 7: Cierre del descriptor.

020 - Clase 02-09 / src / main / webapp / index.html

Qué es y para qué sirve: Página de entrada con enlaces a los dos ejemplos de la clase.

Cómo defenderlo / qué decir: Es HTML puro: sirve como menú para llegar a las demostraciones.

```

1 | <!DOCTYPE html>
2 | <html lang="en">
3 | <head>
4 |   <meta charset="UTF-8">
5 |   <title>Clase 02-0</title>
6 | </head>

```

Líneas 1 a 6: DOCTYPE HTML5, head con charset y título.

```

7 | <body>
8 |   <div>
9 |     <a href="inicio.jsp">Uso de include</a>
10 |     <br>
11 |     <a href="session.jsp">Session</a>
12 |   </div>
13 | </body>
14 | </html>

```

Líneas 7 a 14: Dos enlaces **<a href>**: a inicio.jsp («Uso de include») y a session.jsp («Session»), separados por un **
**.

020 - Clase 02-09 / src / main / webapp / common / navbar.jsp

Qué es y para qué sirve: Fragmento reutilizable: la **barra de navegación** de Bootstrap. Se incluye desde otras páginas.

Cómo defenderlo / qué decir: Es el archivo que recibe los **parámetros** del jsp:include: **classActive** (se imprime dentro del class del enlace «Inicio») e **isContacto** (decide si se marca «Contacto» como activo). Así una única barra sirve a todas las páginas y resalta la opción actual. **Observación:** como se incluye dentro de otra página, este archivo no debería tener sus propias etiquetas **<html>**, **<head>** y **<body>**: genera HTML anidado (los navegadores lo toleran, pero no es válido). Además, si el parámetro classActive no llega, la expresión imprime «null» en el atributo class.

```

1 | <!--
2 |   Created by IntelliJ IDEA.
3 |   User: usal
4 |   Date: 2/9/26
5 |   Time: 7:20 p. m.
6 |   To change this template use File | Settings | File Templates.
7 | -->

```

Líneas 1 a 7: Comentario JSP generado por el IDE.

```

8 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>

```

Línea 8: Directiva **page**: tipo de contenido y codificación UTF-8.

```

9 | <html>
10 | <head>
11 |   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
12 |     integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
13 | </head>

```

Líneas 9 a 12: Abre html y head con el **link a Bootstrap** (CDN).

```

13 | <%
14 |   String classActive = request.getParameter("classActive");
15 | %>
16 |

```

Líneas 13 a 16: **Scriptlet** que lee con **request.getParameter("classActive")** el parámetro enviado por jsp:param desde la página que incluye.

```

17 | <body>
18 |   <nav class="navbar navbar-expand-lg navbar-dark bg-primary mb-4">
19 |     <div class="container">

```

Líneas 17 a 19: Abre el body y la etiqueta **nav** de Bootstrap (navbar-dark bg-primary: barra azul con letras claras).

```

20 |       <a class="navbar-brand" href="inicio.jsp">Portal USAL 2026</a>
21 |       <div class="navbar-nav">

```

Líneas 20 a 21: Marca del sitio «Portal USAL 2026» (enlace a inicio) y contenedor navbar-nav del menú.

```

22 |         <a class="nav-link <%= classActive %%" href="inicio.jsp">Inicio</a>
23 |       <%

```

Líneas 22 a 23: Enlace «Inicio»: una **expresión** imprime el valor de classActive (por ejemplo «active») dentro del atributo class, para resaltarlo.

```

24 |         String active = request.getParameter("isContacto");
25 |
26 |
27 |
28 |         if(active != null && active.equals("true")){
29 |             out.println("<a class=\"nav-link active\" href=\"contacto.jsp\">Contacto</a>");
30 |         } else {
31 |             out.println("<a class=\"nav-link\" href=\"contacto.jsp\">Contacto</a>");
32 |         }
33 |     }
34 | }
35 |

```

Líneas 24 a 35: Lee el parámetro isContacto. Si viene en «true» imprime con **out.println** el enlace «Contacto» con la clase **active**; si no, el enlace normal. (Las comillas dobles internas se escapan con \".)

```

36 | </nav>
37 | </body>
38 | </html>

```

Líneas 36 a 38: Cierra las etiquetas nav, body y html.

020 - Clase 02-09 / src / main / webapp / inicio.jsp

Qué es y para qué sirve: Página «Inicio»: incluye la barra de navegación con **jsp:include** y le pasa el parámetro classActive.

Cómo defenderlo / qué decir: Compara las dos formas de include en el propio código: queda **comentada** la versión estática (<%@ include file=...%>) y está activa la **dinámica** (jsp:include), que permite pasar parámetros con jsp:param. También se ve que el HTML de la barra original se movió a navbar.jsp (queda comentado como recuerdo). **Detalle:** como esta página no carga Bootstrap, los estilos le llegan por el link que está dentro de navbar.jsp.

```

1 | <html>
2 | <head>
3 |   <title>Inicio</title>
4 | </head>

```

Líneas 1 a 4: Apertura de html, head y título.

```

5 | <body class="bg-light">
6 |
7 |   <!--<nav class="navbar navbar-expand-lg navbar-dark bg-primary mb-4">
8 |     <div class="container">
9 |       <a class="navbar-brand" href="inicio.jsp">Portal USAL 2026</a>
10 |       <div class="navbar-nav">
11 |         <a class="nav-link active" href="inicio.jsp">Inicio</a>
12 |         <a class="nav-link" href="contacto.jsp">Contacto</a>
13 |       </div>
14 |     </div>
15 |   </nav-->
16 |

```

Líneas 5 a 16: Body con fondo gris claro de Bootstrap (bg-light). El bloque siguiente es un **comentario JSP** con el HTML de la barra que antes estaba aquí y ahora se reutiliza.

```

17 |   <!--Include estatico-->
18 |   <!--   <%@ include file="common/navbar.jsp"%>-->

```

Líneas 17 a 18: Comentarios: «Include estático» y la directiva **<%@ include file="common/navbar.jsp"%>** desactivada. El estático copia el archivo en tiempo de traducción y no admite parámetros.

```

19 |   <jsp:include page="common/navbar.jsp">
20 |     <jsp:param name="classActive" value="active"/>
21 |   </jsp:include>
22 |

```

Líneas 19 a 22: **Include dinámico:** en cada petición ejecuta common/navbar.jsp e inserta su resultado. Con **<jsp:param name="classActive" value="active"/>** le envía el parámetro que marca «Inicio» como opción activa.

```

23 |     <div class="container">
24 |         <h1>Pagina de inicio</h1>
25 |         <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Alias assumenda debitis optio quia quod ratione
           |         4 rem saepe vel vero voluptate! Asperiores deserunt illum laudantium numquam quia, quo repellendus velit
           |         4 voluptates!</p>
26 |     </div>
27 |
28 |
29 | </body>
30 | </html>

```

Líneas 23 a 30: Contenido de la página: un título y un párrafo de texto de relleno (lorem ipsum). Cierra body y html.

020 - Clase 02-09 / src / main / webapp / contacto.jsp

Qué es y para qué sirve: Página «Contacto»: igual que inicio pero con otro parámetro (isContacto).

Cómo defenderlo / qué decir: Mismo include dinámico, con otro parámetro: así la barra marca como activo «Contacto». Es la prueba de que jsp:include permite que **el mismo fragmento se comporte distinto** según quién lo incluya.

```

1 | <html>
2 | <head>
3 |     <title>Contacto</title>
4 | </head>

```

Líneas 1 a 4: Apertura de html, head y título.

```

5 | <body class="bg-light">
6 |
7 |     <!-- <nav class="navbar navbar-expand-lg navbar-dark bg-primary mb-4">
8 |         <div class="container">
9 |             <a class="navbar-brand" href="inicio.jsp">Portal USAL 2026</a>
10 |             <div class="navbar-nav">
11 |                 <a class="nav-link " href="inicio.jsp">Inicio</a>
12 |                 <a class="nav-link active" href="contacto.jsp">Contacto</a>
13 |             </div>
14 |         </div>
15 |
16 |     </nav>-->

```

Líneas 5 a 16: Body con fondo claro y la barra original comentada.

```

17 |     <!--Include dinamico-->
18 |     <jsp:include page="common/navbar.jsp">
19 |         <jsp:param name="isContacto" value="true"/>
20 |     </jsp:include>
21 |

```

Líneas 17 a 21: Comentario «Include dinámico» y el **jsp:include** de common/navbar.jsp con el parámetro **isContacto = true**.

```

22 |     <div class="container">
23 |         <h1>Pagina de contacto</h1>
24 |         <p>Contactese con mail@mail.com</p>
25 |     </div>
26 | </body>
27 | </html>

```

Líneas 22 a 27: Contenido: título «Página de contacto» y un correo de ejemplo. Cierra body y html.

020 - Clase 02-09 / src / main / webapp / session.jsp

Qué es y para qué sirve: Ejemplo de **Session**: un contador que se guarda en la sesión del usuario.

Cómo defenderlo / qué decir: Muestra el ciclo: **getAttribute** (leer) → incrementar → **setAttribute** (guardar). **Atención:** al final del scriptlet el código hace **removeAttribute** y **session.invalidate()**; como se ejecutan en cada petición, el contador **siempre muestra 1** (nunca llega a acumular). Esas dos líneas están para mostrar cómo se borra un dato y cómo se destruye la sesión; para ver un contador que sube hay que comentarlas. Como además cada petición destruye la sesión, el **Session ID** que se muestra cambia en cada visita (cada vez se crea una sesión nueva).

```

1 | <%--
2 |   Created by IntelliJ IDEA.
3 |   User: usal
4 |   Date: 2/9/26
5 |   Time: 7:57 p. m.
6 |   To change this template use File | Settings | File Templates.
7 | --%>

```

Líneas 1 a 7: Comentario JSP del IDE.

```

8 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>

```

Línea 8: Directiva page (UTF-8).

```

9 | <html>
10 | <head>
11 |   <title>Session en JSP</title>
12 |   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
13 |     integrity="sha384-sRIL4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
14 | </head>

```

Líneas 9 a 13: Apertura de html, head con título y Bootstrap.

```

14 | <body class="bg-light p-5">
15 |   <div class="container">
16 |     <div class="card shadow border-danger">
17 |       <div class="card-header bg-danger text-white">
18 |         <h4 class="mb-0">Ejemplo de contador</h4>
19 |       </div>
20 |     </div>

```

Líneas 14 a 20: Body con padding y una tarjeta (card) de Bootstrap con encabezado rojo «Ejemplo de contador».

```

21 |       <div class="card-body text-center mb-1">
22 |         <%

```

Líneas 21 a 22: Cuerpo de la tarjeta. Comienza el **scriptlet**.

```

23 |           int contador = 0;
24 |
25 |           Integer contadorSession = (Integer) session.getAttribute("miContador");
26 |

```

Líneas 23 a 26: Variable local para el valor a mostrar. Luego lee el atributo **miContador** de la sesión con **getAttribute** y lo castea a Integer (devuelve null la primera vez).

```

27 |           session.setMaxInactiveInterval(5000);
28 |

```

Líneas 27 a 28: Define que la sesión expira tras **5000 segundos** de inactividad.

```

29 |           if(contadorSession != null && contadorSession > 0) {
30 |             contadorSession = contadorSession + 1;
31 |             contador = contadorSession;
32 |             session.setAttribute("miContador", contadorSession);
33 |           } else {
34 |
35 |             contador = contador + 1;
36 |             session.setAttribute("miContador", contador);
37 |           }
38 |

```

Líneas 29 a 38: Si ya había contador, le suma 1 y lo **guarda de nuevo** con **setAttribute**. Si no (primera visita), arranca en 1 y también lo guarda.

```

39 |           /*Borro valor del atributo*/
40 |           session.removeAttribute("miContador");
41 |           /*Elimina la sesion completa*/
42 |           session.invalidate();
43 |         %>

```

Líneas 39 a 43: **removeAttribute("miContador")** borra el dato y **session.invalidate()** destruye la sesión completa. Por ejecutarse siempre, el contador no avanza (ver defensa).

```
44 |         <p class="fs-4">Valor del contador: </p>
45 |         <h4>
46 |             <%= contador %>
47 |         </h4>
48 |         <p>Session ID : <%= session.getId() %></p>
```

Líneas 44 a 48: Fuera del scriptlet: muestra con **expresiones** el valor del contador y el **Session ID** (session.getId()).

```
49 |             <a class="btn btn-outline-danger" href="session.jsp">Incrementar +1</a>
50 |         </div>
51 |     </div>
52 |
53 | </body>
54 | </html>
```

Líneas 49 a 54: Botón que vuelve a pedir la misma página (nueva petición) para «incrementar».

Resumen para estudiar

- Include **estático** (<%@ include %>): se copia al traducir, sin parámetros. Include **dinámico** (<jsp:include>): se ejecuta en cada petición y admite <jsp:param>.
- HTTP no tiene estado; la **Session** lo simula: un objeto por usuario en el servidor + cookie JSESSIONID en el navegador.
- HttpSession: setAttribute, getAttribute, removeAttribute, invalidate, getId, isNew, setMaxInactiveInterval.
- Scopes: page < request < session < application.

Preguntas típicas

¿Cuál es la diferencia entre include estático y dinámico?

El estático inserta el código del otro archivo al traducir el JSP (un solo servlet); el dinámico lo ejecuta en cada petición, permite pasar parámetros y puede dar resultados distintos cada vez.

¿Dónde se guardan los datos de la Session?

En el servidor. Al cliente solo viaja el identificador JSESSIONID en una cookie.

¿Cómo se cierra una sesión?

Con session.invalidate(), que destruye la sesión y sus atributos; además conviene expirar las cookies relacionadas.